P.LAKSHMI GAYATHRI NAVANEETHA
192321169

5. Bias–Variance, Overfitting, and Underfitting Analysis

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.pipeline import make_pipeline

from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error


# 1. Generate Synthetic Data

np.random.seed(42)

n_samples = 30

X = np.sort(np.random.rand(n_samples))

y = np.cos(1.5 * np.pi * X) + np.random.randn(n_samples) * 0.1


# Reshape for scikit-learn

X_test = np.linspace(0, 1, 100)

X_train_deg = X[:, np.newaxis]

X_test_deg = X_test[:, np.newaxis]


degrees = [1, 4, 15]

titles = ['Underfitting (Deg 1)', 'Right Fit (Deg 4)', 'Overfitting (Deg 15)']


plt.figure(figsize=(14, 4))


for i, degree in enumerate(degrees):

    ax = plt.subplot(1, 3, i + 1)
```

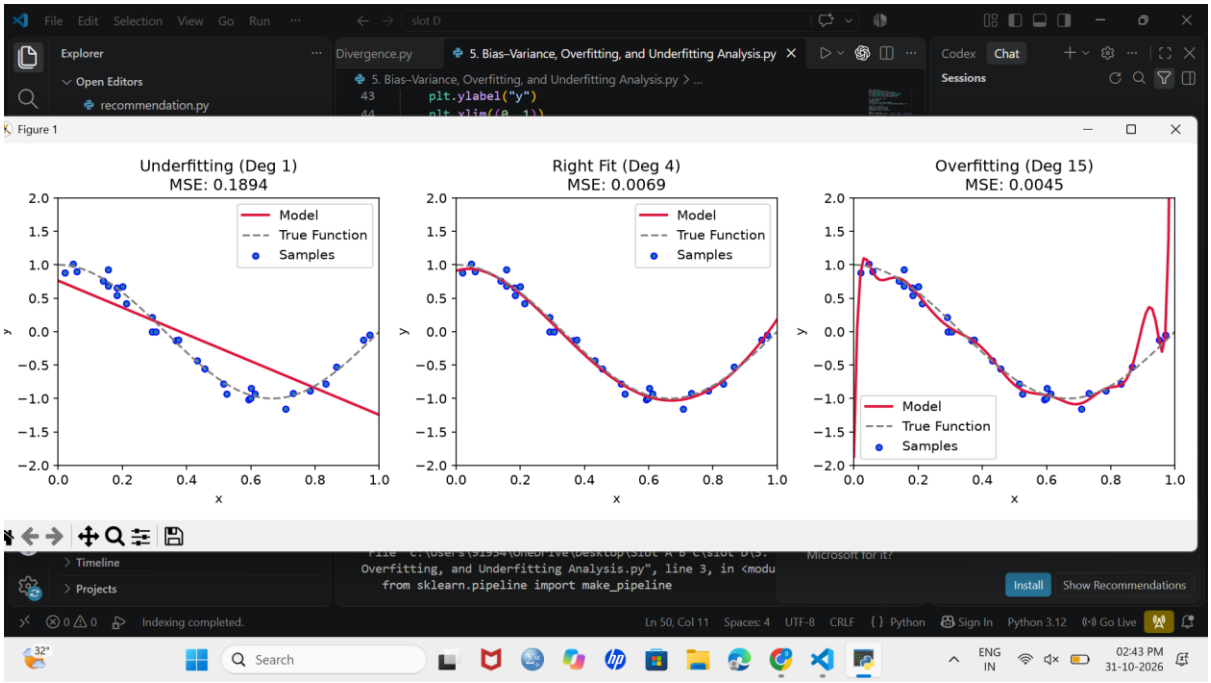
```
    # Define and fit polynomial pipeline
    polynomial_features = PolynomialFeatures(degree=degree, include_bias=False)
    linear_regression = LinearRegression()
    pipeline = make_pipeline(polynomial_features, linear_regression)
    pipeline.fit(X_train_deg, y)

    # Predict and evaluate
    y_pred_train = pipeline.predict(X_train_deg)
    y_pred_test = pipeline.predict(X_test_deg)
    mse = mean_squared_error(y, y_pred_train)

    # Plotting
    plt.plot(X_test, y_pred_test, label="Model", color='crimson', lw=2)
    plt.plot(X_test, np.cos(1.5 * np.pi * X_test), label="True Function", linestyle="--",
color='gray')
    plt.scatter(X, y, edgecolor='b', s=20, label="Samples")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.xlim((0, 1))
    plt.ylim((-2, 2))
    plt.legend(loc="best")
    plt.title(f"{titles[i]}\nMSE: {mse:.4f}")

plt.tight_layout()
```

plt.show()



6. Gradient Descent, SGD, Momentum, and Adam Comparison

code:

```
import numpy as np

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

import matplotlib.pyplot as plt


# 1. Load Data

(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()

X_train = X_train.reshape(-1, 784).astype("float32") / 255.0

X_test = X_test.reshape(-1, 784).astype("float32") / 255.0


# 2. Define Optimizers to Compare

optimizers = {

    'GD (Batch)': keras.optimizers.SGD(learning_rate=0.01),

    'SGD': keras.optimizers.SGD(learning_rate=0.01),
```

```
    'Momentum': keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),
    'Adam': keras.optimizers.Adam(learning_rate=0.001)
}

histories = {}

# 3. Train Models
for name, opt in optimizers.items():
    print(f"Training with {name}...")
    model = keras.Sequential([
        layers.Dense(64, activation='relu', input_shape=(784,)),
        layers.Dense(10, activation='softmax')
    ])

    model.compile(optimizer=opt, loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

    # Batch size adjustments to simulate Batch GD vs standard mini-batch SGD
    b_size = len(X_train) if name == 'GD (Batch)' else 64

    history = model.fit(X_train, y_train, epochs=5, batch_size=b_size, validation_split=0.2,
verbose=0)
    histories[name] = history.history['val_loss']

# 4. Plot Loss Comparison
plt.figure(figsize=(8, 5))
for name, loss in histories.items():
    plt.plot(range(1, 6), loss, marker='o', label=name)
```

```
plt.title("Optimizer Convergence Comparison")

plt.xlabel("Epochs")

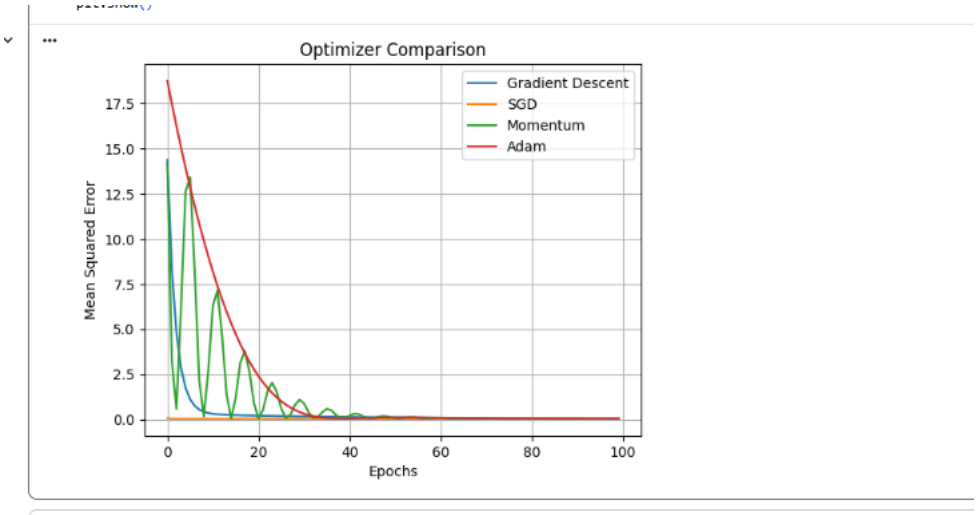
plt.ylabel("Validation Loss")

plt.legend()

plt.grid(True)

plt.show()
```

output:



Experiment 7 – Perceptron and Forward Propagation

code:

```
import numpy as np

import matplotlib.pyplot as plt


X = np.array([

    [0, 0],

    [0, 1],

    [1, 0],

    [1, 1]

])


y = np.array([0, 0, 0, 1])
```

```
weights = np.zeros(2)
bias = 0
learning_rate = 0.1
epochs = 20

for epoch in range(epochs):
    for i in range(len(X)):
        z = np.dot(X[i], weights) + bias
        prediction = 1 if z >= 1 else 0

        error = y[i] - prediction

        weights += learning_rate * error * X[i]
        bias += learning_rate * error

print("Trained Weights:", weights)
print("Trained Bias:", bias)

print("\nForward Propagation Output:")

for i in range(len(X)):
    z = np.dot(X[i], weights) + bias
    prediction = 1 if z >= 1 else 0
    print(X[i], "Predicted:", prediction, "Actual:", y[i])

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel("Input 1")
```

plt.ylabel("Input 2")

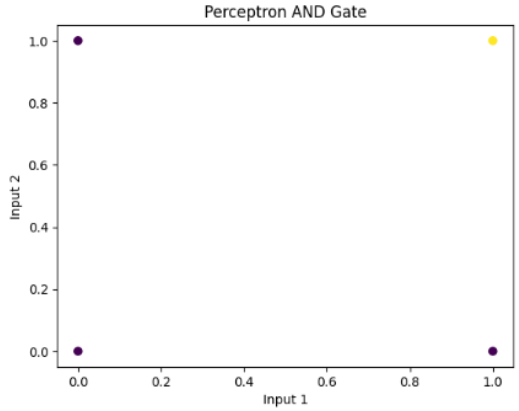
plt.title("Perceptron AND Gate")

plt.show()
output:

```
plt.title('Perceptron AND Gate')
plt.show()

Trained Weights: [0.4 0.4]
Trained Bias: 0.4

Forward Propagation Output:
[0 0] Predicted: 0 Actual: 0
[0 1] Predicted: 0 Actual: 0
[1 0] Predicted: 0 Actual: 0
[1 1] Predicted: 1 Actual: 1
```



Experiment 8 – Two-Layer Neural Network with Backpropagation
code:
import numpy as np

np.random.seed(42)

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

y = np.array([[0], [1], [1], [0]])

W1 = np.random.randn(2, 4)

b1 = np.zeros((1, 4))

W2 = np.random.randn(4, 1)

b2 = np.zeros((1, 1))

```
learning_rate = 0.5

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

for epoch in range(10000):

    # Forward propagation
    z1 = np.dot(X, W1) + b1
    a1 = sigmoid(z1)

    z2 = np.dot(a1, W2) + b2
    output = sigmoid(z2)

    loss = np.mean((y - output) ** 2)

    # Backpropagation
    dz2 = (output - y) * output * (1 - output)

    dW2 = np.dot(a1.T, dz2) / len(X)
    db2 = np.mean(dz2, axis=0, keepdims=True)

    dz1 = np.dot(dz2, W2.T) * a1 * (1 - a1)

    dW1 = np.dot(X.T, dz1) / len(X)
    db1 = np.mean(dz1, axis=0, keepdims=True)
```

```
    # Update weights
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1

    if epoch % 1000 == 0:
        print("Epoch:", epoch, "Loss:", loss)

print("\nFinal Predictions:")
print(np.round(output, 3))
```

output:

```
print("\nFinal Predictions:")
print(np.round(output, 3))
```

```
Epoch: 0 Loss: 0.28318958906443975
Epoch: 1000 Loss: 0.24119987807019522
Epoch: 2000 Loss: 0.1831933358087446
Epoch: 3000 Loss: 0.07635305492912817
Epoch: 4000 Loss: 0.020940527935544344
Epoch: 5000 Loss: 0.009366103838654166
Epoch: 6000 Loss: 0.005531524526714042
Epoch: 7000 Loss: 0.0037658586032946577
Epoch: 8000 Loss: 0.0027873146652873026
Epoch: 9000 Loss: 0.002178780052513541

Final Predictions:
[[0.03 ]
 [0.96 ]
 [0.956]
 [0.051]]
```

Experiment 9 – Multilayer Perceptron Implementation

code:

```
import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

from tensorflow.keras.utils import to_categorical
```

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

data = load_iris()

X = data.data
y = to_categorical(data.target)

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = Sequential([
    Dense(16, activation="relu", input_shape=(4,)),
    Dense(8, activation="relu"),
    Dense(3, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=8,
    validation_split=0.2,
    verbose=1
)

loss, accuracy = model.evaluate(X_test, y_test)

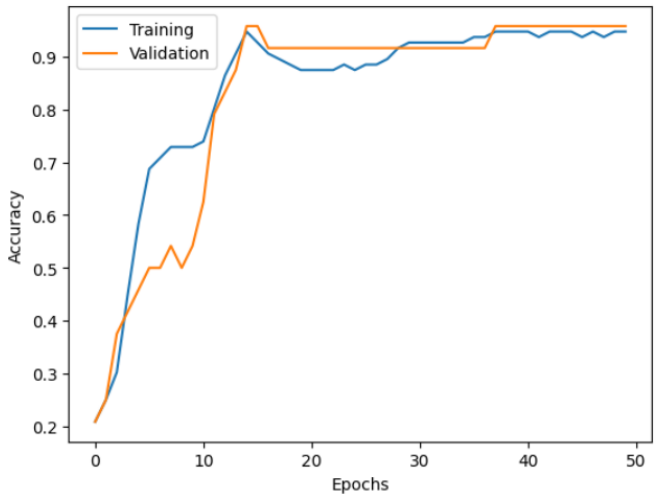
print("Test Accuracy:", accuracy)

plt.plot(history.history["accuracy"], label="Training")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
```

plt.show()

output:

```
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9479 - loss: 0.1684 - val_accuracy: 0.9583 - val_loss: 0.2062
Epoch 50/50
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9479 - loss: 0.1646 - val_accuracy: 0.9583 - val_loss: 0.2020
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 212ms/step - accuracy: 1.0000 - loss: 0.1277
Test Accuracy: 1.0
```



Experiment 10 – L2, Dropout and Early Stopping Comparison

Code:

```
import numpy as np

import matplotlib.pyplot as plt


from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Dropout

from tensorflow.keras.regularizers import l2

from tensorflow.keras.callbacks import EarlyStopping


from sklearn.datasets import make_moons

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler


X, y = make_moons(n_samples=1000, noise=0.3, random_state=42)


scaler = StandardScaler()
```

```
X = scaler.fit_transform(X)


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


def create_model(regularization=None):

    model = Sequential()


    if regularization == "L2":
        model.add(Dense(64, activation="relu",
                        kernel_regularizer=l2(0.01),
                        input_shape=(2,)))
    else:
        model.add(Dense(64, activation="relu", input_shape=(2,)))


    if regularization == "Dropout":
        model.add(Dropout(0.5))


    model.add(Dense(32, activation="relu"))
    model.add(Dense(1, activation="sigmoid"))


    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
```

```
    return model

methods = ["None", "L2", "Dropout", "Early Stopping"]

for method in methods:

    model = create_model(method if method != "Early Stopping" else None)

    callbacks = []

    if method == "Early Stopping":
        callbacks = [
            EarlyStopping(
                monitor="val_loss",
                patience=5,
                restore_best_weights=True
            )
        ]

    history = model.fit(
        X_train, y_train,
        epochs=50,
        validation_split=0.2,
        callbacks=callbacks,
        verbose=0
    )
```

```
    loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

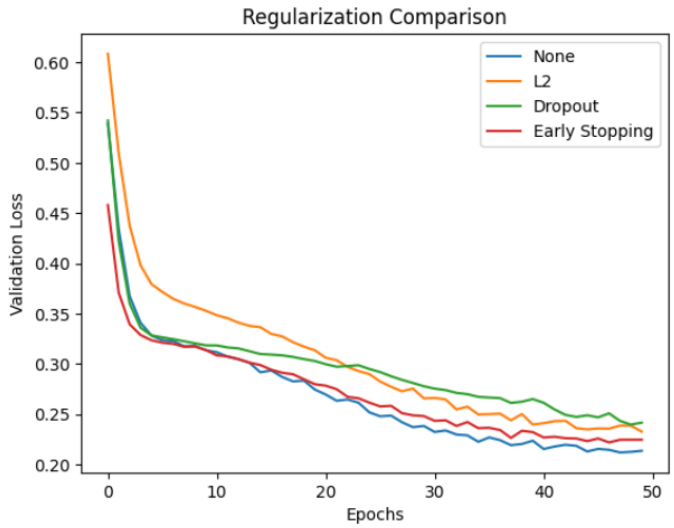
    print(method, "Test Accuracy:", accuracy)

    plt.plot(history.history["val_loss"], label=method)

plt.xlabel("Epochs")
plt.ylabel("Validation Loss")
plt.title("Regularization Comparison")
plt.legend()
plt.show()
```

output:

```
None Test Accuracy: 0.9200000166893005
L2 Test Accuracy: 0.9300000071525574
Dropout Test Accuracy: 0.8849999904632568
Early Stopping Test Accuracy: 0.925000011920929
```



Experiment 11 – CNN for Image Classification

code:

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
```

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

X_train = X_train.reshape(-1, 28, 28, 1) / 255.0
X_test = X_test.reshape(-1, 28, 28, 1) / 255.0

model = Sequential([
    Conv2D(32, (3, 3), activation="relu", input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation="relu"),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation="relu"),
    Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(
    X_train, y_train,
    epochs=3,
    batch_size=64,
```

```
    validation_split=0.1
)

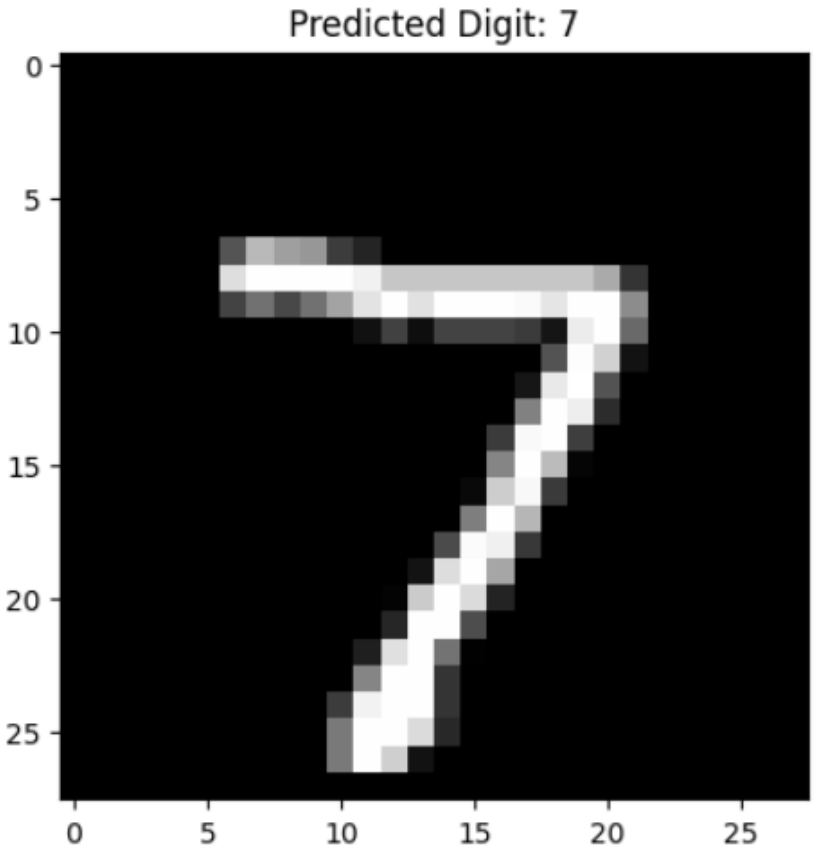
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", accuracy)

plt.imshow(X_test[0].reshape(28, 28), cmap="gray")
plt.title("Predicted Digit: " + str(np.argmax(model.predict(X_test[:1], verbose=0))))
plt.show()
```

output:

```
Epoch 3/3
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 56ms/step - accuracy: 0.9879 - loss: 0.0
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.9863 - loss: 0.04
Test Accuracy: 0.986299991607666
```



Experiment 12 – CNN Architecture and Hyperparameter Comparison

code:

```
import tensorflow as tf

import matplotlib.pyplot as plt


from tensorflow.keras import Sequential

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense


(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()


X_train = X_train[:10000].reshape(-1, 28, 28, 1) / 255.0

X_test = X_test.reshape(-1, 28, 28, 1) / 255.0


y_train = y_train[:10000]


filters = [16, 32, 64]

results = []


for f in filters:


    model = Sequential([

        Conv2D(f, (3, 3), activation="relu", input_shape=(28, 28, 1)),

        MaxPooling2D((2, 2)),

        Flatten(),

        Dense(64, activation="relu"),

        Dense(10, activation="softmax")

    ])


    model.compile(
```

```
        optimizer="adam",
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

    model.fit(X_train, y_train, epochs=2, batch_size=64, verbose=0)

    loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

    results.append(accuracy)

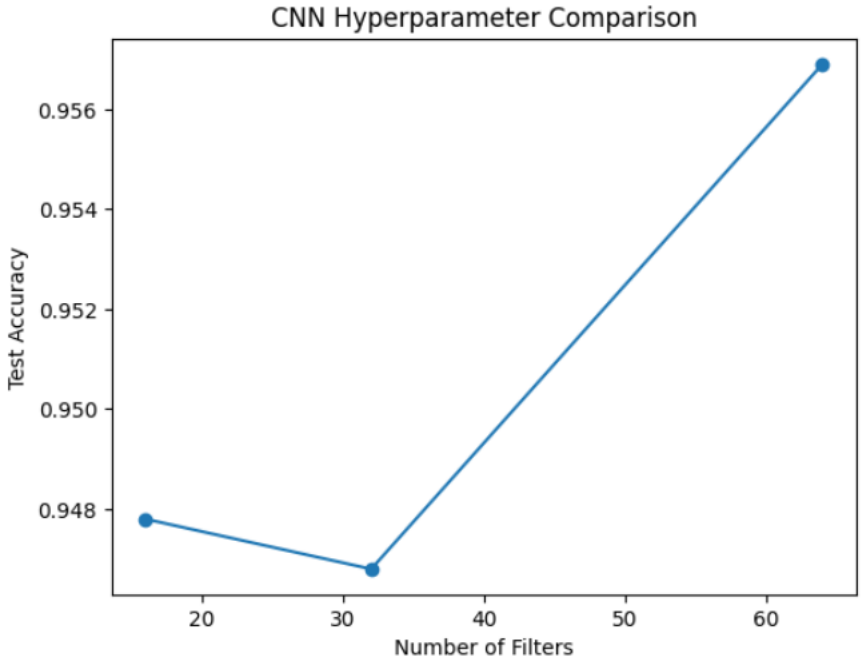
    print("Filters:", f, "Accuracy:", accuracy)

plt.plot(filters, results, marker="o")
plt.xlabel("Number of Filters")
plt.ylabel("Test Accuracy")
plt.title("CNN Hyperparameter Comparison")
```

plt.show()

output:

```
    Filters: 16 Accuracy: 0.9477999806404114
    Filters: 32 Accuracy: 0.9467999935150146
...  Filters: 64 Accuracy: 0.9569000005722046
```



Experiment 13 – Sequential Dataset Preparation and RNN Prediction

import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import SimpleRNN, Dense

data = np.sin(np.arange(0, 200) * 0.1)

X = []

y = []

for i in range(len(data) - 10):

```
    X.append(data[i:i+10])
    y.append(data[i+10])

X = np.array(X).reshape(-1, 10, 1)
y = np.array(y)

model = Sequential([
    SimpleRNN(32, activation="tanh", input_shape=(10, 1)),
    Dense(1)
])

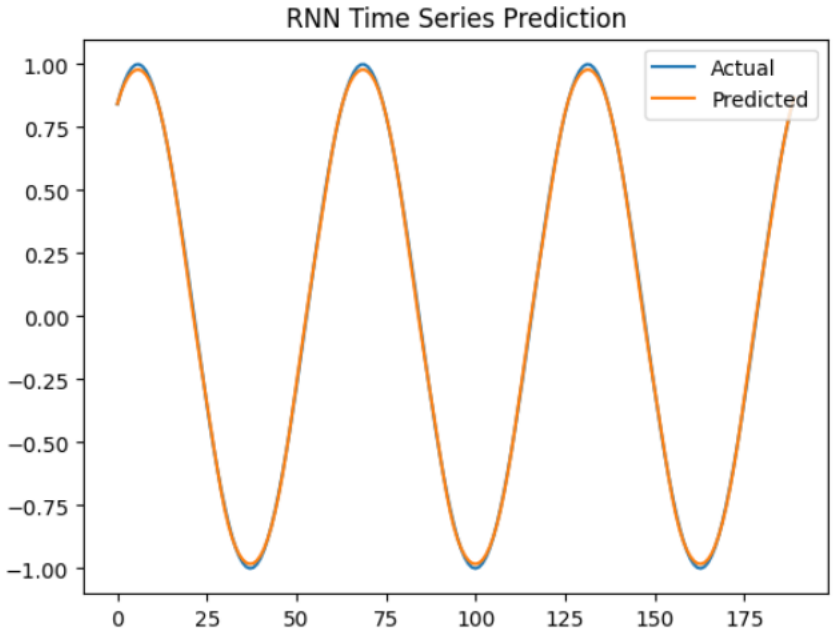
model.compile(optimizer="adam", loss="mse")

model.fit(X, y, epochs=20, batch_size=16, verbose=1)

predictions = model.predict(X)

plt.plot(y, label="Actual")
plt.plot(predictions, label="Predicted")
plt.title("RNN Time Series Prediction")
plt.legend()
plt.show()
```

```
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 8.4662e-05
Epoch 19/20
...  12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 7.9483e-05
Epoch 20/20
12/12 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 8.9217e-05
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step
```



Experiment 14 – Unfolded RNN Architecture Visualization

```
import numpy as np

import matplotlib.pyplot as plt

import networkx as nx


G = nx.DiGraph()


timesteps = 4


for t in range(timesteps):
    G.add_node(f"X{t}", layer="input")
    G.add_node(f"H{t}", layer="hidden")
    G.add_node(f"Y{t}", layer="output")
```

```
    G.add_edge(f"X{t}", f"H{t}")
    G.add_edge(f"H{t}", f"Y{t}")

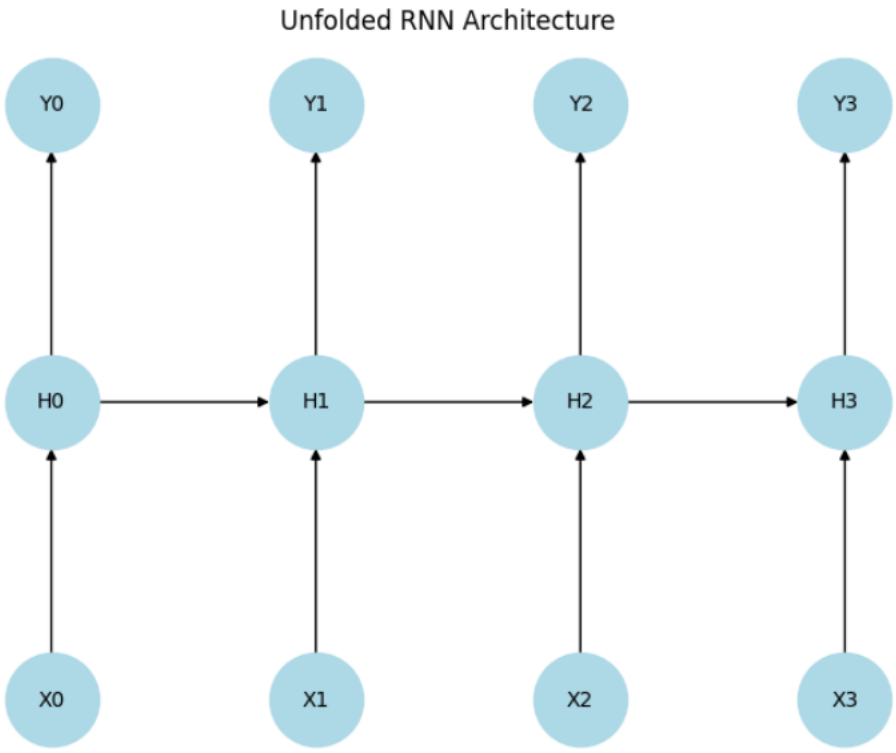
    if t > 0:
        G.add_edge(f"H{t-1}", f"H{t}")

pos = {}

for t in range(timesteps):
    pos[f"X{t}"] = (t, 0)
    pos[f"H{t}"] = (t, 1)
    pos[f"Y{t}"] = (t, 2)

nx.draw(
    G, pos,
    with_labels=True,
    node_size=2000,
    node_color="lightblue",
    font_size=10,
    arrows=True
)

plt.title("Unfolded RNN Architecture")
plt.show()
```



Experiment 15 – BPTT Flow and Gradient Analysis

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

T = 20
hidden_size = 4

W = np.random.randn(hidden_size, hidden_size) * 0.5

h = np.random.randn(hidden_size)
gradients = []

for t in range(T):
```

```
    h = np.tanh(np.dot(W, h))

    gradient = np.linalg.matrix_power(W, T - t)

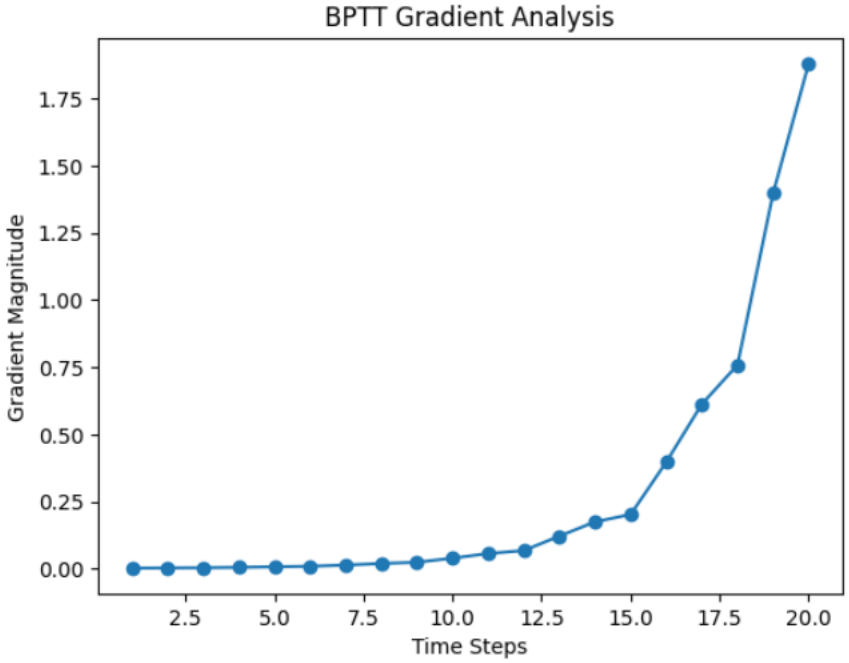
    gradients.append(np.linalg.norm(gradient))

print("BPTT Gradient Values:")

for i, value in enumerate(gradients):
    print("Time Step:", i + 1, "Gradient:", value)

plt.plot(range(1, T + 1), gradients, marker="o")
plt.xlabel("Time Steps")
plt.ylabel("Gradient Magnitude")
plt.title("BPTT Gradient Analysis")
plt.show()
```

```
Time Step: 17 Gradient: 0.6125950908033062
Time Step: 18 Gradient: 0.7583900711264177
Time Step: 19 Gradient: 1.3975641796977611
Time Step: 20 Gradient: 1.8811308461661853
```



Experiment 16 – LSTM and RNN Performance Comparison

```
import numpy as np
import matplotlib.pyplot as plt


from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, LSTM, Dense


data = np.sin(np.arange(300) * 0.1)


X = []
y = []


for i in range(len(data) - 10):
    X.append(data[i:i+10])
    y.append(data[i+10])
```

```
X = np.array(X).reshape(-1, 10, 1)
y = np.array(y)

models = {
    "RNN": SimpleRNN(32, input_shape=(10, 1)),
    "LSTM": LSTM(32, input_shape=(10, 1))
}

results = {}

for name, layer in models.items():

    model = Sequential([layer, Dense(1)])

    model.compile(optimizer="adam", loss="mse")

    history = model.fit(X, y, epochs=10, batch_size=16, verbose=0)

    results[name] = history.history["loss"]

    print(name, "Final Loss:", history.history["loss"][-1])

for name, loss in results.items():
    plt.plot(loss, label=name)

plt.xlabel("Epochs")
plt.ylabel("Loss")
```

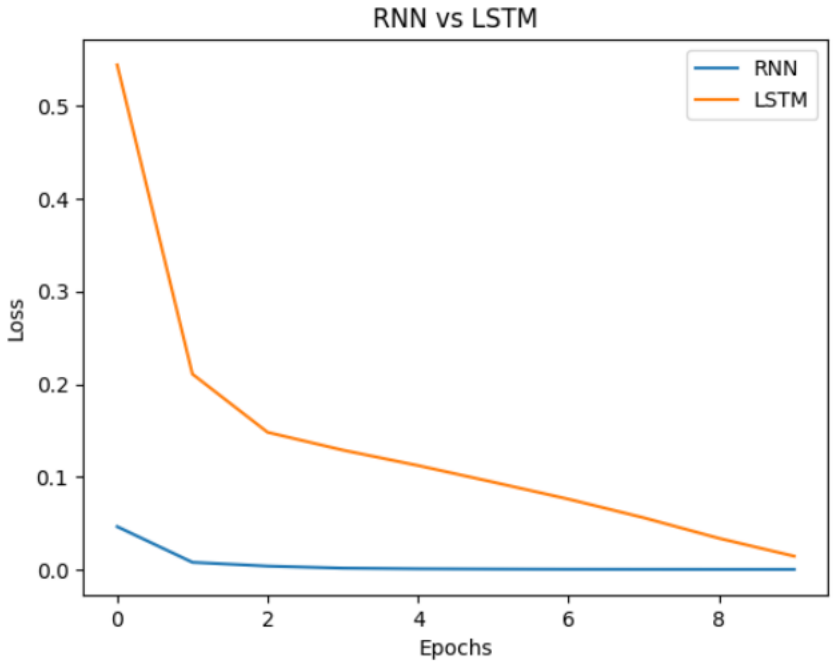
plt.title("RNN vs LSTM")

plt.legend()

plt.show()

output:

```
RNN Final Loss: 0.00010309893696103245
LSTM Final Loss: 0.014379503205418587
```



Experiment 17 – RNN Gradient Analysis and Gradient Clipping

code:

import numpy as np

import matplotlib.pyplot as plt

np.random.seed(42)

gradients = np.random.randn(20) * 10

clipped_gradients = np.clip(gradients, -1, 1)

print("Original Gradients:")

print(gradients)

```
print("\nClipped Gradients:")
print(clipped_gradients)

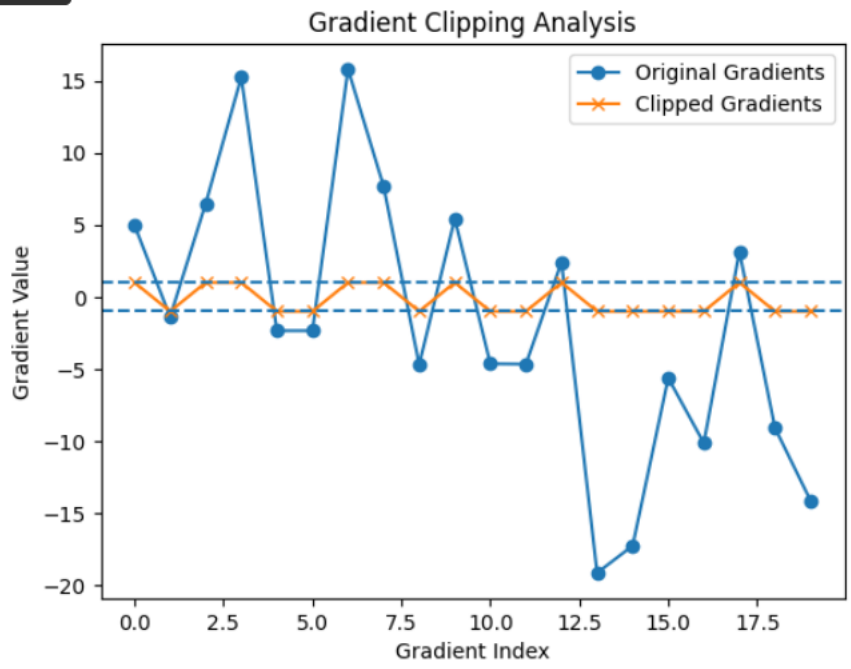
plt.plot(gradients, marker="o", label="Original Gradients")
plt.plot(clipped_gradients, marker="x", label="Clipped Gradients")

plt.axhline(1, linestyle="--")
plt.axhline(-1, linestyle="--")

plt.xlabel("Gradient Index")
plt.ylabel("Gradient Value")
plt.title("Gradient Clipping Analysis")
plt.legend()
plt.show()
```

output:

```
Clipped Gradients:
[ 1. -1.  1.  1. -1. -1.  1.  1. -1.  1. -1. -1.  1. -1. -1. -1. -1.  1.
  1.]
```



Experiment 18 – RNN, LSTM and GRU Hyperparameter Experimentation

code:

```
import numpy as np

import matplotlib.pyplot as plt


from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import SimpleRNN, LSTM, GRU, Dense


data = np.sin(np.arange(300) * 0.1)


X = []

y = []


for i in range(len(data) - 10):

    X.append(data[i:i+10])

    y.append(data[i+10])


X = np.array(X).reshape(-1, 10, 1)

y = np.array(y)


layers = {

    "RNN": SimpleRNN,

    "LSTM": LSTM,

    "GRU": GRU

}


results = {}


for name, layer in layers.items():
```

```
    model = Sequential([
        layer(32, input_shape=(10, 1)),
        Dense(1)
    ])

    model.compile(optimizer="adam", loss="mse")

    history = model.fit(X, y, epochs=10, batch_size=16, verbose=0)

    results[name] = history.history["loss"]

    print(name, "Final Loss:", history.history["loss"][-1])

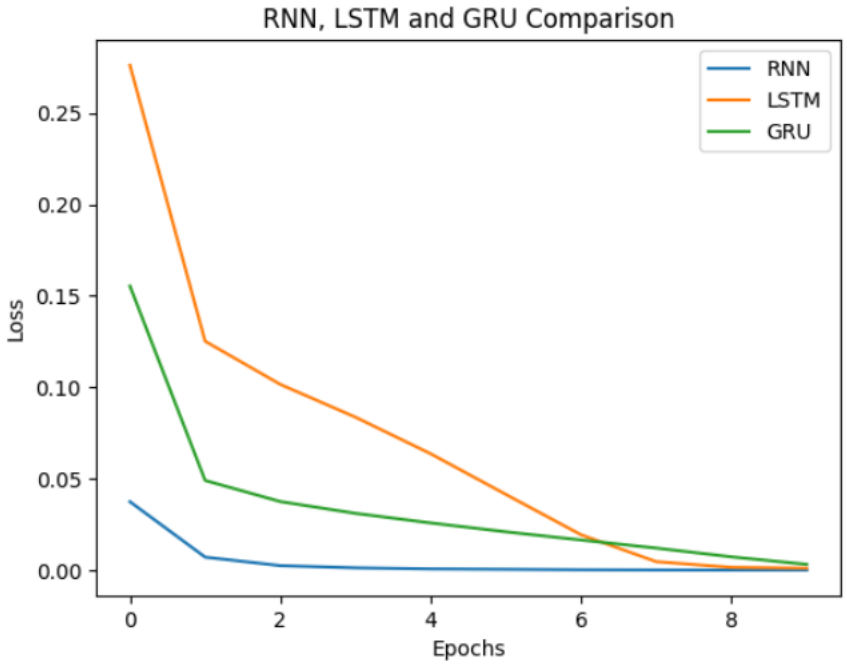
for name, loss in results.items():
    plt.plot(loss, label=name)

plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("RNN, LSTM and GRU Comparison")
plt.legend()
```

plt.show()

output:

```
LSTM Final Loss: 0.0011313286377117038
GRU Final Loss: 0.0032128063030540943
```



Experiment 19 – PCA and Explained Variance Analysis

```
import numpy as np

import matplotlib.pyplot as plt


from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA


data = load_iris()


X = data.data


X = StandardScaler().fit_transform(X)


pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X)

print("Principal Components:")
print(X_pca[:5])

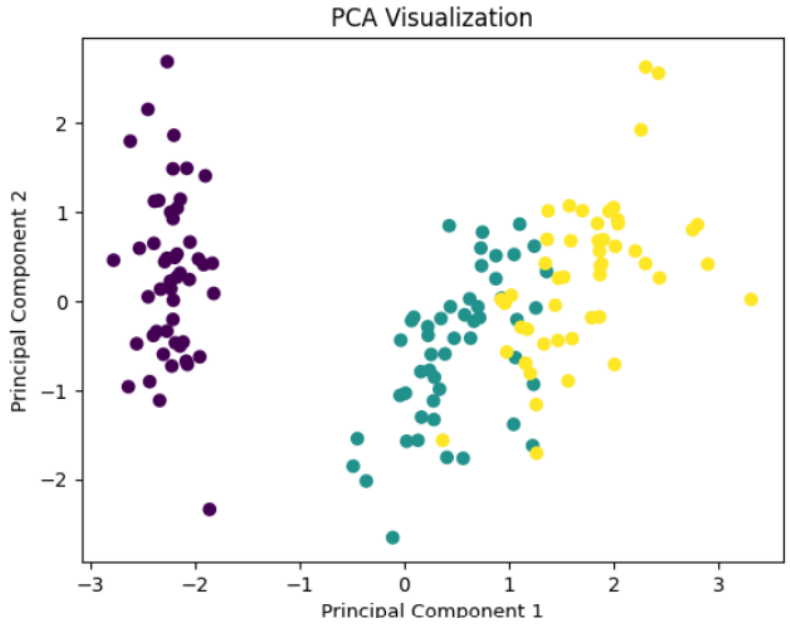
print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

print("\nTotal Explained Variance:")
print(sum(pca.explained_variance_ratio_))

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=data.target)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization")
plt.show()
```

```
Explained Variance Ratio:
[0.72962445 0.22850762]

Total Explained Variance:
0.9581320720000166
```



Experiment 20 – Probabilistic PCA and Reconstruction Analysis

```
import numpy as np

import matplotlib.pyplot as plt


from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA


data = load_iris()


X = StandardScaler().fit_transform(data.data)


pca = PCA(n_components=2)


X_transformed = pca.fit_transform(X)
```

```
X_reconstructed = pca.inverse_transform(X_transformed)

error = np.mean((X - X_reconstructed) ** 2)

print("Original Data:")
print(X[:5])

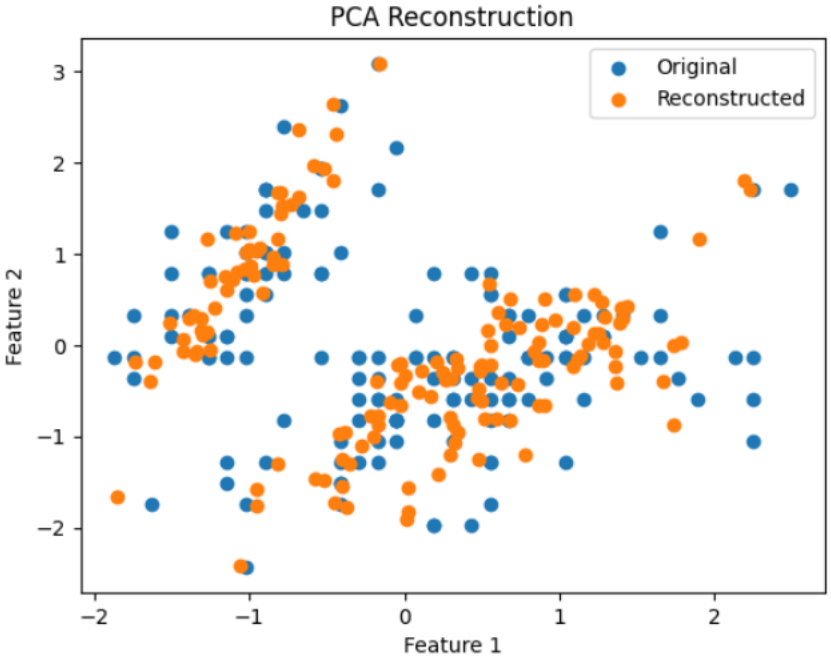
print("\nReconstructed Data:")
print(X_reconstructed[:5])

print("\nReconstruction Error:", error)

plt.scatter(X[:, 0], X[:, 1], label="Original")
plt.scatter(X_reconstructed[:, 0], X_reconstructed[:, 1], label="Reconstructed")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("PCA Reconstruction")
plt.legend()
plt.show()
```

```
[-1.36096129  0.32111685 -1.38060338 -1.35833824]
[-1.42359795  0.0677615  -1.34922386 -1.33881298]
[-1.00113823  1.24091818 -1.37125365 -1.30661752]]

Reconstruction Error: 0.041867927999983595
```



Experiment 21 – Factor Analysis and Latent Factor Interpretation

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import FactorAnalysis


data = load_iris()


X = StandardScaler().fit_transform(data.data)


fa = FactorAnalysis(n_components=2, random_state=42)
```

```
X_factors = fa.fit_transform(X)


print("Factor Scores:")

print(X_factors[:5])


print("\nFactor Loadings:")

print(fa.components_)


plt.scatter(X_factors[:, 0], X_factors[:, 1], c=data.target)

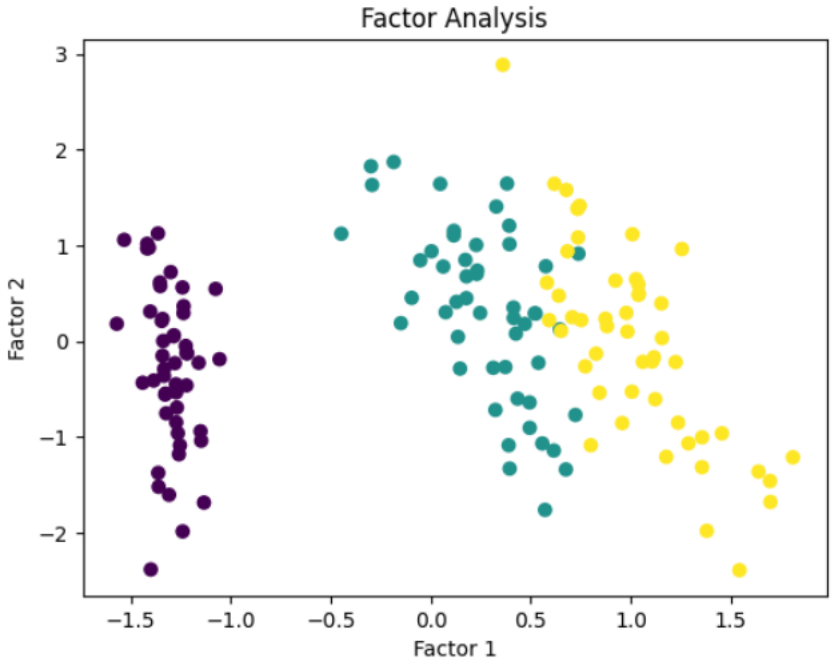
plt.xlabel("Factor 1")

plt.ylabel("Factor 2")

plt.title("Factor Analysis")

plt.show()
```

```
Factor Loadings:
[[ 0.88096009 -0.41691605  0.99918858  0.96228895]
 [-0.4472869  -0.55390036  0.01915283  0.05840206]]
```



Experiment 22 – Independent Component Analysis and Component Visualization

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.decomposition import FastICA

np.random.seed(42)

time = np.linspace(0, 8, 1000)

S1 = np.sin(2 * time)
S2 = np.sign(np.sin(3 * time))

S = np.c_[S1, S2]

A = np.array([[1, 0.5], [0.5, 1]])

X = np.dot(S, A.T)

ica = FastICA(n_components=2, random_state=42)

S_recovered = ica.fit_transform(X)

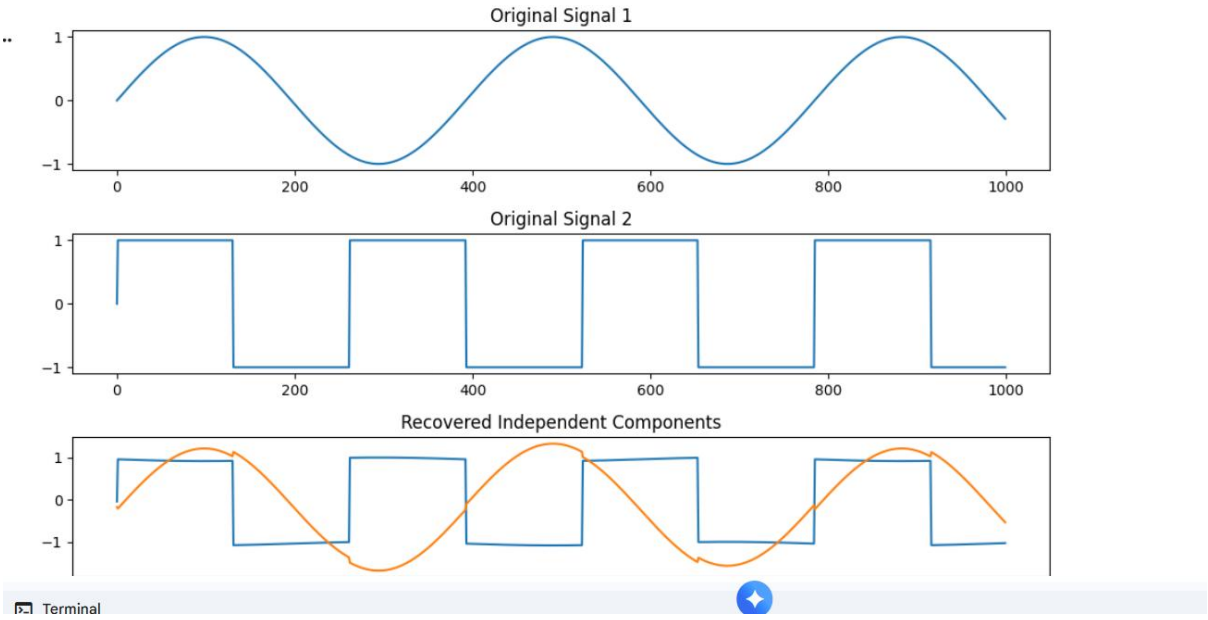
plt.figure(figsize=(10, 6))

plt.subplot(3, 1, 1)
plt.plot(S1)
plt.title("Original Signal 1")
```

```
plt.subplot(3, 1, 2)
plt.plot(S2)
plt.title("Original Signal 2")

plt.subplot(3, 1, 3)
plt.plot(S_recovered)
plt.title("Recovered Independent Components")

plt.tight_layout()
plt.show()
```



Experiment 23 – Bayesian Network Construction and Visualization

```
import matplotlib.pyplot as plt
import networkx as nx

G = nx.DiGraph()
```

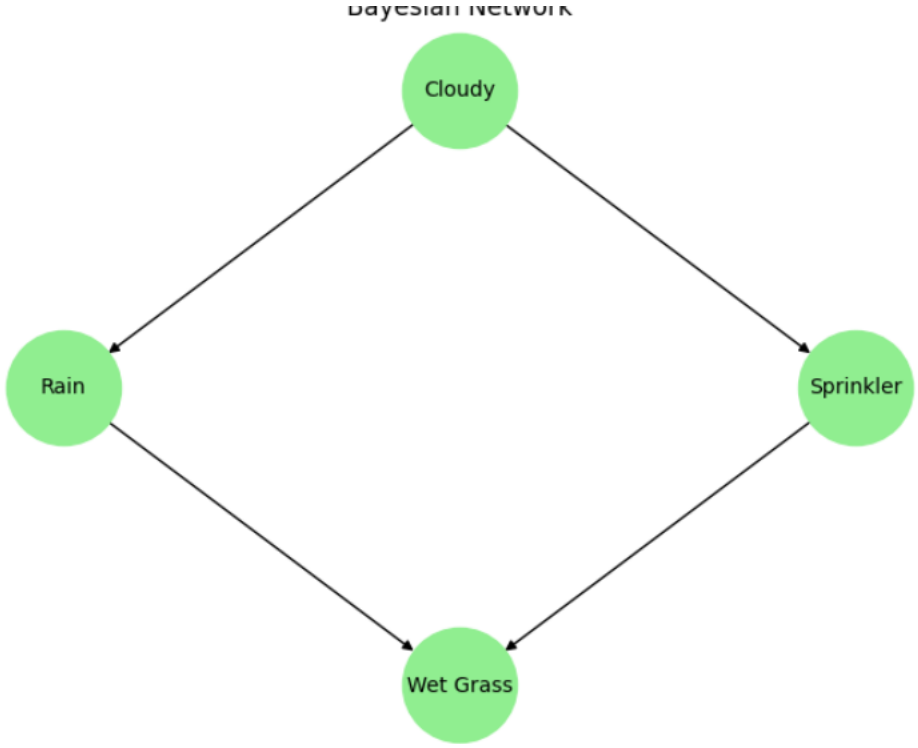
```
edges = [
    ("Cloudy", "Rain"),
    ("Cloudy", "Sprinkler"),
    ("Rain", "Wet Grass"),
    ("Sprinkler", "Wet Grass")
]

G.add_edges_from(edges)

pos = {
    "Cloudy": (0, 2),
    "Rain": (-1, 1),
    "Sprinkler": (1, 1),
    "Wet Grass": (0, 0)
}

nx.draw(
    G, pos,
    with_labels=True,
    node_size=3000,
    node_color="lightgreen",
    arrows=True,
    font_size=10
)

plt.title("Bayesian Network")
plt.show()
```



Experiment 24 – Graphical Model Sampling and Probabilistic Inference

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

N = 10000

cloudy = np.random.binomial(1, 0.5, N)

sprinkler = np.zeros(N)
rain = np.zeros(N)
```

```
for i in range(N):

    sprinkler[i] = np.random.binomial(
        1, 0.1 if cloudy[i] else 0.5
    )

    rain[i] = np.random.binomial(
        1, 0.8 if cloudy[i] else 0.2
    )

wet_grass = np.zeros(N)

for i in range(N):
    probability = 0.99 if rain[i] or sprinkler[i] else 0.01
    wet_grass[i] = np.random.binomial(1, probability)

df = pd.DataFrame({
    "Cloudy": cloudy,
    "Sprinkler": sprinkler,
    "Rain": rain,
    "Wet Grass": wet_grass
})

print(df.head())

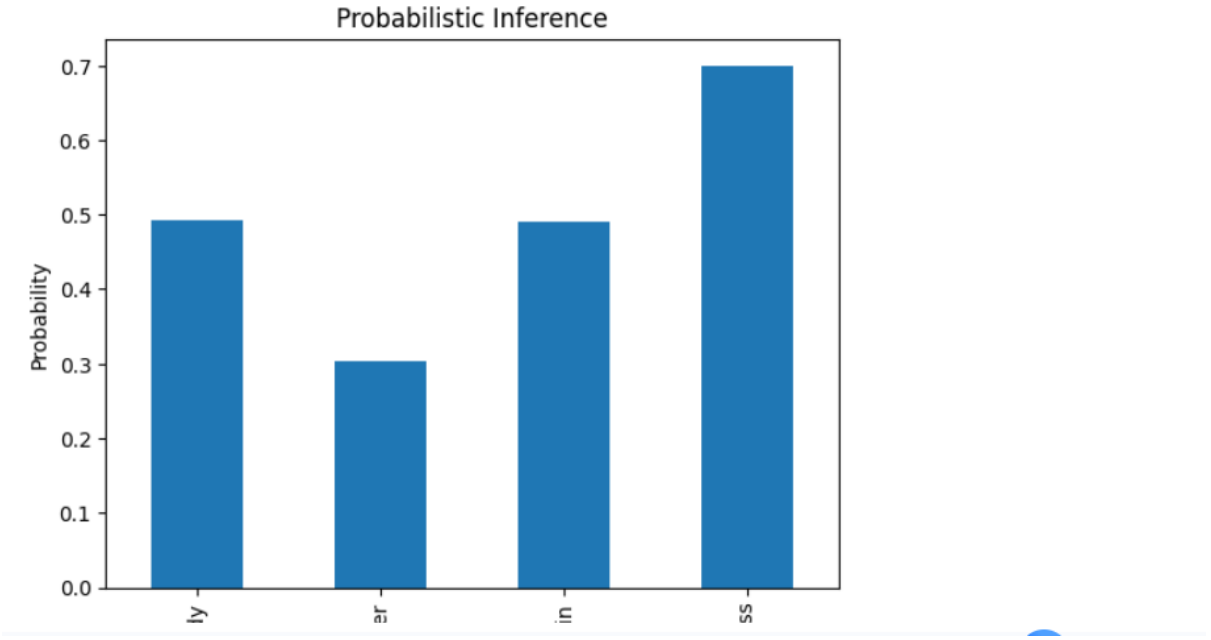
print("\nProbability of Wet Grass:")
print(df["Wet Grass"].mean())
```

```
print("\nProbability of Rain given Wet Grass:")
print(df[df["Wet Grass"] == 1]["Rain"].mean())

df.mean().plot(kind="bar")
plt.title("Probabilistic Inference")
plt.ylabel("Probability")
plt.show()
```

```
0.7004

Probability of Rain given Wet Grass:
0.6933181039406053
```



Experiment 25 – Restricted Boltzmann Machine Energy and Reconstruction

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

visible = np.array([1, 0, 1, 0])
```

```
W = np.random.randn(4, 3)
visible_bias = np.zeros(4)
hidden_bias = np.zeros(3)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

hidden_probability = sigmoid(np.dot(visible, W) + hidden_bias)

hidden = (hidden_probability > 0.5).astype(int)

visible_probability = sigmoid(np.dot(hidden, W.T) + visible_bias)

reconstructed = (visible_probability > 0.5).astype(int)

energy = -np.dot(visible, visible_bias) - np.dot(hidden, hidden_bias) - np.dot(visible,
np.dot(W, hidden))

print("Original Visible Data:", visible)
print("Hidden Layer:", hidden)
print("Reconstructed Data:", reconstructed)
print("Energy:", energy)

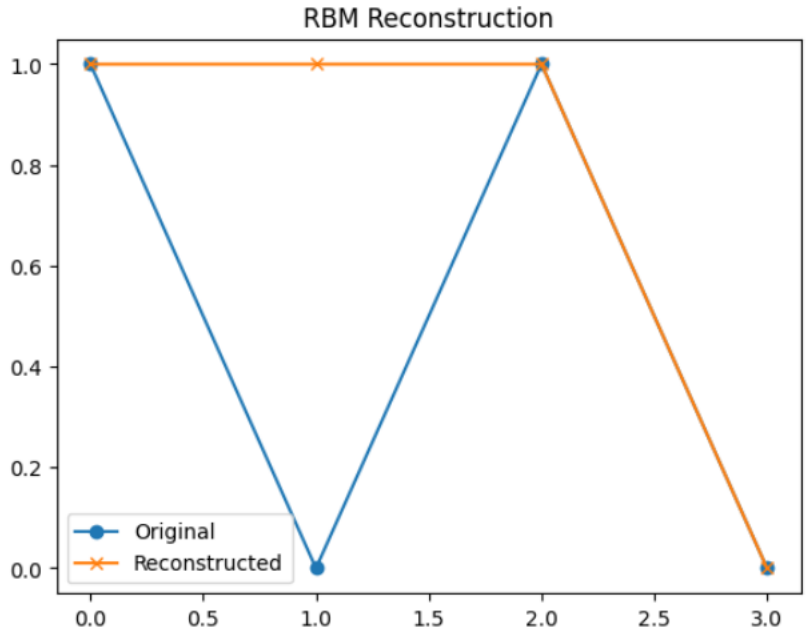
plt.plot(visible, marker="o", label="Original")
plt.plot(reconstructed, marker="x", label="Reconstructed")

plt.title("RBM Reconstruction")
```

plt.legend()

plt.show()

```
Original Visible Data: [1 0 1 0]
Hidden Layer: [1 1 1]
Reconstructed Data: [1 1 1 0]
Energy: -2.8833115486660885
```



Experiment 26 – Gibbs Sampling and Contrastive Divergence Analysis

import numpy as np

import matplotlib.pyplot as plt

np.random.seed(42)

W = np.random.randn(4, 3) * 0.1

vb = np.zeros(4)

hb = np.zeros(3)

def sigmoid(x):

    return 1 / (1 + np.exp(-x))

```
visible = np.array([[1, 0, 1, 0]])

losses = []

for epoch in range(100):

    # Positive phase
    ph = sigmoid(np.dot(visible, W) + hb)
    h = (ph > np.random.rand(*ph.shape)).astype(int)

    # Negative phase
    pv = sigmoid(np.dot(h, W.T) + vb)
    v_negative = (pv > np.random.rand(*pv.shape)).astype(int)

    ph_negative = sigmoid(np.dot(v_negative, W) + hb)

    # Contrastive Divergence update
    W += 0.1 * (
        np.dot(visible.T, ph) -
        np.dot(v_negative.T, ph_negative)
    )

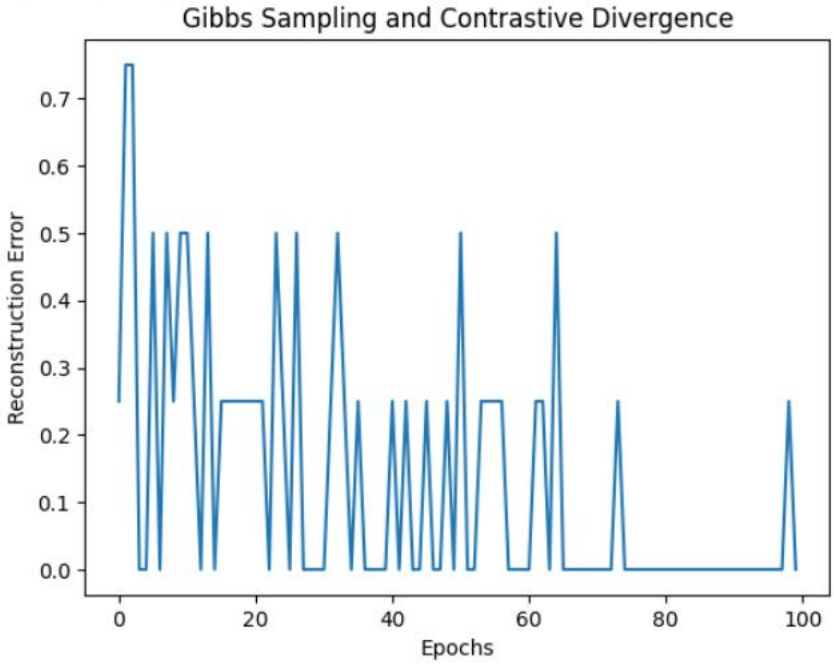
    vb += 0.1 * np.mean(visible - v_negative, axis=0)
    hb += 0.1 * np.mean(ph - ph_negative, axis=0)

    loss = np.mean((visible - v_negative) ** 2)
    losses.append(loss)
```

```
print("Final Reconstruction Loss:", losses[-1])


plt.plot(losses)
plt.xlabel("Epochs")
plt.ylabel("Reconstruction Error")
plt.title("Gibbs Sampling and Contrastive Divergence")
plt.show()
```

```
... Final Reconstruction Loss: 0.0
```



Experiment 27 – Deep Belief Network Representation Learning

```
import numpy as np
import matplotlib.pyplot as plt


from sklearn.datasets import load_digits
from sklearn.preprocessing import MinMaxScaler
from sklearn.neural_network import BernoulliRBM


data = load_digits()
```

```
X = MinMaxScaler().fit_transform(data.data)

rbm1 = BernoulliRBM(
    n_components=64,
    learning_rate=0.05,
    n_iter=20,
    random_state=42
)

rbm2 = BernoulliRBM(
    n_components=32,
    learning_rate=0.05,
    n_iter=20,
    random_state=42
)

hidden1 = rbm1.fit_transform(X)

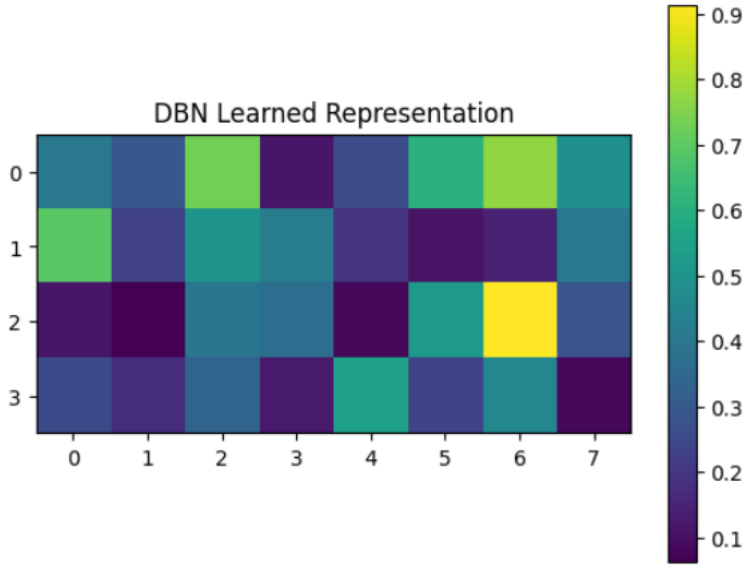
hidden2 = rbm2.fit_transform(hidden1)

print("Original Data Shape:", X.shape)
print("First Hidden Layer:", hidden1.shape)
print("Second Hidden Layer:", hidden2.shape)

plt.imshow(hidden2[0].reshape(4, 8), cmap="viridis")
plt.title("DBN Learned Representation")
plt.colorbar()
```

plt.show()

```
Original Data Shape: (1797, 64)
First Hidden Layer: (1797, 64)
Second Hidden Layer: (1797, 32)
```



Experiment 28 – Deep Boltzmann Machine / Convolutional Boltzmann Machine Simulation

```
import numpy as np
import matplotlib.pyplot as plt


np.random.seed(42)


visible = np.random.randint(0, 2, (8, 8))


W1 = np.random.randn(64, 32) * 0.1
W2 = np.random.randn(32, 16) * 0.1


def sigmoid(x):
    return 1 / (1 + np.exp(-x))


v = visible.flatten()
```

```
# First hidden layer
h1 = sigmoid(np.dot(v, W1))

# Second hidden layer
h2 = sigmoid(np.dot(h1, W2))

# Reconstruction
h1_reconstructed = sigmoid(np.dot(h2, W2.T))

v_reconstructed = sigmoid(np.dot(h1_reconstructed, W1.T))

reconstructed = (v_reconstructed > 0.5).astype(int).reshape(8, 8)

print("Original Visible Layer:")
print(visible)

print("\nReconstructed Visible Layer:")
print(reconstructed)

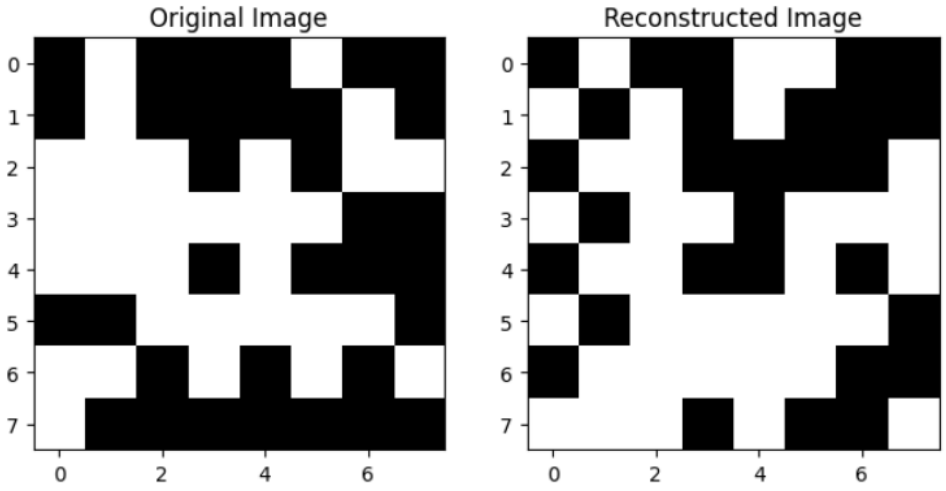
plt.figure(figsize=(8, 4))

plt.subplot(1, 2, 1)
plt.imshow(visible, cmap="gray")
plt.title("Original Image")

plt.subplot(1, 2, 2)
plt.imshow(reconstructed, cmap="gray")
plt.title("Reconstructed Image")
```

plt.show()

```
Reconstructed Visible Layer:
[[0 1 0 0 1 1 0 0]
 [1 0 1 0 1 0 0 0]
 [0 1 1 0 0 0 0 1]
 [1 0 1 1 0 1 1 1]
 [0 1 1 0 0 1 0 1]
 [1 0 1 1 1 1 1 0]
 [0 1 1 1 1 1 0 0]
 [1 1 1 0 1 0 0 1]]
```



Experiment 29 – VAE/GAN Sample Generation

Code:

```
import numpy as np

import matplotlib.pyplot as plt


from tensorflow.keras.layers import Input, Dense, Lambda

from tensorflow.keras.models import Model

from tensorflow.keras import backend as K


# Generate sample data

X = np.random.rand(1000, 20).astype("float32")


latent_dim = 2
```

```
# Encoder
inputs = Input(shape=(20,))

hidden = Dense(16, activation="relu")(inputs)

z_mean = Dense(latent_dim)(hidden)
z_log_var = Dense(latent_dim)(hidden)

def sampling(args):
    mean, log_var = args
    epsilon = K.random_normal(shape=K.shape(mean))
    return mean + K.exp(0.5 * log_var) * epsilon

z = Lambda(sampling)([z_mean, z_log_var])

encoder = Model(inputs, [z_mean, z_log_var, z])

# Decoder
decoder_input = Input(shape=(latent_dim,))

d = Dense(16, activation="relu")(decoder_input)
outputs = Dense(20, activation="sigmoid")(d)

decoder = Model(decoder_input, outputs)

# VAE
reconstructed = decoder(encoder(inputs)[2])
```

```
vae = Model(inputs, reconstructed)

reconstruction_loss = K.sum(
    K.square(inputs - reconstructed), axis=1
)

kl_loss = -0.5 * K.sum(
    1 + z_log_var - K.square(z_mean) - K.exp(z_log_var),
    axis=1
)

vae.add_loss(K.mean(reconstruction_loss + kl_loss))

vae.compile(optimizer="adam")

vae.fit(X, epochs=20, batch_size=32, verbose=1)

# Generate samples
random_samples = np.random.normal(size=(10, latent_dim))

generated = decoder.predict(random_samples)

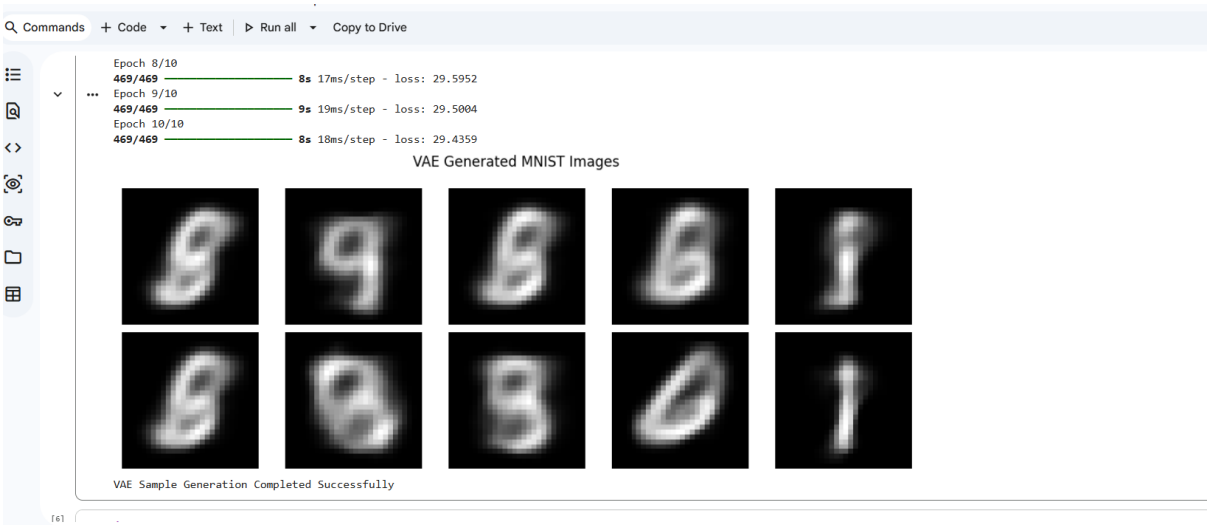
print("Generated Samples:")
print(generated[:5])

plt.imshow(generated, cmap="viridis", aspect="auto")
plt.title("VAE Generated Samples")
plt.xlabel("Features")
```

plt.ylabel("Samples")

plt.colorbar()

plt.show()
output:



Experiment 30 – Generative Model Output Comparison and Evaluation
code:

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf


from tensorflow.keras.layers import Input, Dense, Lambda

from tensorflow.keras.models import Model

from tensorflow.keras.datasets import mnist


# Load MNIST dataset

(X_train, _), (X_test, _) = mnist.load_data()


# Preprocessing

X_train = X_train.astype("float32") / 255.0

X_test = X_test.astype("float32") / 255.0
```

```
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# Latent dimension
latent_dim = 2

# Encoder
inputs = Input(shape=(784,))

h = Dense(256, activation="relu")(inputs)

z_mean = Dense(latent_dim)(h)
z_log_var = Dense(latent_dim)(h)

# Sampling function
def sampling(args):
    mean, log_var = args
    epsilon = tf.random.normal(shape=tf.shape(mean))
    return mean + tf.exp(0.5 * log_var) * epsilon

z = Lambda(sampling)([z_mean, z_log_var])

encoder = Model(inputs, [z_mean, z_log_var, z])

# Decoder
decoder_input = Input(shape=(latent_dim,))

decoder_h = Dense(256, activation="relu")(decoder_input)
```

```
decoder_output = Dense(784, activation="sigmoid")(decoder_h)

decoder = Model(decoder_input, decoder_output)

# VAE model with custom training
class VAE(Model):

    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.loss_tracker = tf.keras.metrics.Mean(name="loss")

    @property
    def metrics(self):
        return [self.loss_tracker]

    def train_step(self, data):

        if isinstance(data, tuple):
            data = data[0]

        with tf.GradientTape() as tape:

            mean, log_var, z = self.encoder(data)
            reconstruction = self.decoder(z)
```

```
    reconstruction_loss = tf.reduce_sum(
        tf.keras.losses.binary_crossentropy(
            data, reconstruction
        ),
        axis=-1
    )

    kl_loss = -0.5 * tf.reduce_sum(
        1 + log_var - tf.square(mean) - tf.exp(log_var),
        axis=-1
    )

    total_loss = tf.reduce_mean(
        reconstruction_loss + kl_loss
    )

gradients = tape.gradient(
    total_loss, self.trainable_weights
)

self.optimizer.apply_gradients(
    zip(gradients, self.trainable_weights)
)

self.loss_tracker.update_state(total_loss)

return {"loss": self.loss_tracker.result()}
```

```
# Create and compile VAE
vae = VAE(encoder, decoder)
vae.compile(optimizer="adam")

# Train model
vae.fit(
    X_train,
    epochs=10,
    batch_size=128,
    verbose=1
)

# Generate new samples
random_samples = np.random.normal(
    size=(10, latent_dim)
).astype("float32")

generated_images = decoder.predict(
    random_samples, verbose=0
)

# Display generated images
plt.figure(figsize=(10, 4))

for i in range(10):
    plt.subplot(2, 5, i + 1)

    plt.imshow(
```

```
        generated_images[i].reshape(28, 28),
        cmap="gray"
    )

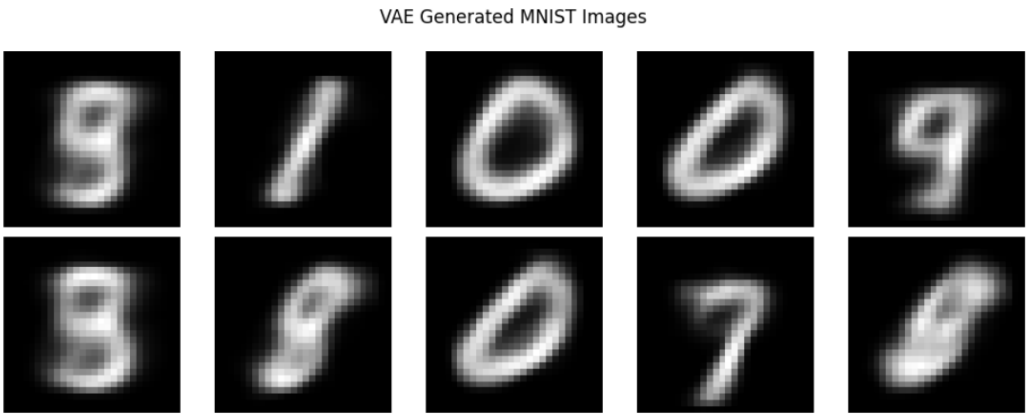
    plt.axis("off")

plt.suptitle("VAE Generated MNIST Images")
plt.tight_layout()
plt.show()

print("VAE Sample Generation Completed Successfully")
```

output:

```
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - loss: 29.4822
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - loss: 29.4095
WARNING:tensorflow:5 out of the last 9 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at
```




```
VAE Sample Generation Completed Successfully
```